

RESEARCH

Open Access



A federated learning and decision transformer-based multi-source data fusion architecture for real-time industrial scheduling dashboards

Mengmeng Su¹ and Man Hu^{1*}

*Correspondence:

Man Hu

Manhuu2026@outlook.com

¹North China Branch of State Grid Corporation of China, Beijing 100053, China

Abstract

With the rapid development of industrial IoT and intelligent scheduling systems, real-time fusion of multi-source heterogeneous data and decision optimization have become critical challenges. Traditional centralized data collection methods face privacy leakage and communication bottlenecks, while federated learning (FL) protects data privacy through distributed model training but has limited temporal decision-making capabilities in dynamic scheduling scenarios. This study combines federated learning with Decision Transformers (DT) to propose a multi-source data fusion architecture for real-time industrial scheduling dashboards, addressing privacy protection and temporal prediction challenges in cross-departmental, cross-regional data collaboration. The system employs a FedAvg-based federated learning framework for local training at 20 distributed nodes and global model aggregation, enhancing model generalization while safeguarding data privacy. Leveraging Decision Transformers' strengths in modeling long-range temporal dependencies, it dynamically optimizes dispatch strategies via trajectory-based sequence learning. Experiments were deployed in a simulated industrial scheduling environment incorporating five heterogeneous data sources (device sensor data, historical scheduling records, environmental parameters, network status data, and user demand data) with 100,000 temporal samples. We compared the proposed method with traditional centralized LSTM, single federated learning, XGBoost, LightGBM, and modern tabular deep learning models. Results demonstrate a 12.7% improvement in scheduling accuracy over the traditional centralized LSTM model, a 63% reduction in cumulative parameter-transmission volume relative to the FedAvg-GRU baseline under the same client participation schedule, and an empirical privacy-attack success rate below 0.05%. Under dynamic load fluctuation scenarios, the single-step inference latency remains below 200 ms, whereas the average time required to complete an end-to-end scheduling cycle is 3.1 min; the system processes more than 2000 real-time data streams per second with an average CPU utilization of 51.6% and a peak memory usage of 8.9 GB. This research offers a novel solution for multi-source data fusion in industrial scheduling dashboards, balancing privacy security with decision efficiency, and validates the effectiveness of integrating federated learning with decision-making time-series models.



© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

Keywords Federated learning, Decision transformer, Multi-source data fusion, Scheduling dashboard, Privacy protection

1 Introduction

With the continuous deepening of industrial intelligentization and the extensive deployment of information infrastructure, multi-source heterogeneous data has shown explosive growth in complex systems. Especially in scheduling scenarios involving cross-domain collaboration and real-time decision-making, the effective fusion and intelligent utilization of massive data have become the key to improving the system's responsiveness and operational robustness [1–3]. When faced with the real-world environment of wide geographic distribution, strict privacy constraints, and dispersed computing resources, the traditional centralized data processing model has gradually revealed bottlenecks, including high communication costs, difficulty breaking down data silos, and increased security risks. Therefore, it is urgent to explore a new architecture that balances privacy protection, collaborative efficiency, and decision-making intelligence [4, 5]. As a distributed machine learning paradigm, federated learning can realize the collaborative optimization of multi-party models without the need to aggregate the original data centrally, providing a feasible path for the joint mining of cross-domain data value. Its decentralized and privacy-friendly characteristics meet the requirements of multi-institutional linkage and the coexistence of sensitive information faced by scheduling dashboards [6]. At the same time, as a type of generative policy network that introduces the idea of sequence modeling into the reinforcement learning framework, Decision Transformer can model long-range dependencies with the help of historical trajectories and task goals, and directly learn a robust decision mapping under offline data conditions, opening up new ideas for the scheduling system to achieve forward-looking planning and adaptive adjustment in an uncertain environment [7, 8]. Organically integrating the two into the multi-source data fusion system of the scheduling dashboard is not only expected to break through the limitations of existing methods in limited data acquisition and insufficient policy generalization, but also can build a new form of intelligent scheduling with global perception, privacy security, and policy foresight [9, 10].

In the application of scheduling dashboards for complex production operations and public service networks, multi-source data often covers multi-modal information such as sensor time-series signals, business management records, environmental perception images, and user interaction logs. There are significant differences in the structural form, spatio-temporal scale, and semantic level among these data sources. The fusion process not only needs to address cross-domain feature alignment and noise suppression, but also maintain the timeliness and consistency of information under dynamically changing working conditions [11–13]. Most traditional fusion mechanisms rely on a central node for unified data cleaning and feature extraction, which easily weakens scalability due to communication overhead and trust issues. They may not be implemented due to data ownership and compliance requirements in cross-institutional collaboration [14]. The introduction of federated learning enables each participant to perform feature encoding and model updates locally and to conduct only limited interactions via encrypted gradients or parameter summaries, thereby forming a globally consistent representation space while ensuring data sovereignty and providing robust support for the semantic

bridging of multi-source heterogeneous components [15]. However, simply relying on federated learning for feature-level collaboration is not enough to directly produce optimized actions for scheduling goals. It still needs to be combined with a decision-making mechanism that supports sequence reasoning and conditional generation [16].

Decision Transformer serializes states, actions, and rewards into a stream of tokens that an autoregressive model can model. It performs conditional generation using the target return and historical context, transforming the policy learning process into the pattern reproduction and generalization of high-quality trajectory data. This offline data-based inverse reinforcement learning method can capture the scheduling wisdom implicit in historical operations without online interaction or a simulation environment, offering natural advantages for rapid visual deduction and interpretable policy backtracking required by the scheduling dashboard. When combined with federated learning, the local experience trajectories of each node can participate in the training of the global model under the privacy protection framework, which not only avoids the leakage risk caused by centralized collection of original decision-making data but also improves the adaptability of the policy to diverse scenarios through the sharing of cross-domain experiences. Further, at the presentation level of the scheduling large screen, the unified situational picture generated by multi-source data fusion and the forward-looking scheduling suggestions output by Decision Transformer can be visually linked, enabling operators to perceive the system evolution trend in an intuitive interface and intervene in key decision-making processes, thus bridging the gap between underlying data intelligence and high-level cognitive control.

Currently, independent research on federated learning and deep reinforcement learning has yielded substantial results. However, in the application scenario of dispatch dashboards—which demands real-time processing, multi-agent collaboration, and visual interaction—the construction of an integrated framework that balances privacy security, cross-domain coordination, and long-term decision-making capabilities remains an exploratory frontier. In this context, multi-source data fusion systems serve not only as technical vehicles for information integration but also as trusted intermediaries bridging distributed sensing and centralized decision-making. Their design must establish a dynamic equilibrium among model learning efficiency, policy robustness, and human-machine collaboration usability.

Recent studies on adaptive machine-learning optimization and hierarchical feature-driven decision-making have shown that robust model performance can be improved when algorithms explicitly consider informative feature selection, heterogeneous data, and dynamically changing operating conditions. This observation is relevant to the present work, where multi-source scheduling data must be fused efficiently while maintaining reliable and privacy-preserving decision performance.

To balance recent application-oriented studies with foundational work, this study is grounded in the original Decision Transformer formulation [17], FedProx for heterogeneous federated optimization [18], and the federated machine-learning framework summarized by Yang et al. [19]. These works provide the theoretical basis for sequence-conditioned decision modeling, non-IID client optimization, and privacy-preserving cross-silo collaboration.

The core contribution of this article is to propose a multi-source data fusion architecture that combines FL and DT, and for the first time achieve long-term scheduling

decisions under privacy protection; Design task adaptive DT parameter allocation strategy to reduce inference delay while ensuring accuracy; Constructing an industrial scheduling simulation environment and verifying its robustness under dynamic loads.

This study aims to design an industrial scheduling multi-source data fusion architecture that balances privacy protection and decision efficiency, and proposes the following hypothesis: the fusion architecture of federated learning and decision Transformer can improve scheduling accuracy while protecting privacy (H1); The task adaptive DT parameter allocation strategy can reduce inference delay (H2) while maintaining accuracy; This architecture has good robustness under dynamic loads and non IID data (H3).

2 Theoretical basis and analysis of relevant technologies

2.1 Basic principles and collaborative mechanism of federated learning

FL uses a distributed training strategy. Each device or client independently trains on its local dataset to enhance privacy protection and avoid sharing raw data [20]. After training, the client uploads the updated version of its local model to the central node, which integrates the update content and optimizes the global model parameters. By storing, processing, and learning data locally and avoiding direct exchanges, privacy protection is improved [21].

Federated learning adopts a collaborative strategy to jointly optimize and build a general machine learning model on the device side. Each device retains its local data, as shown in Fig. 1. The training process is as follows:

The server initializes the global model and distributes it to all clients. This model can obtain basic model parameters through pre-training on the initial global dataset [22]. Each client starts local training from this global model, aiming to lay a unified foundation for subsequent training and ensure that all clients have the same initial parameters and synchronous training [23]. First, pre-process the client's local data without uploading it to the central server to ensure privacy. After receiving the global model, the client iteratively trains the model with its own dataset to optimize the parameters, involving activities such as forward propagation, loss function calculation, back-propagation to calculate gradients, and weight updates [24]. Each client conducts multiple local training rounds, updating the weight parameters in each round to improve the learning ability. The training process is shown in formula (1).

$$w_k^{(t+1)} = w_k^{(t)} - \eta_k \nabla_w L_k(w_k^{(t)}, D_k) \quad (1)$$

Among them, η_k is the learning rate of client k , $L_k(w_k^{(t)}, D_k)$ is the loss function based on the local dataset D_k , and $\nabla_w L_k(w_k^{(t)}, D_k)$ is the gradient of the local loss function with respect to the model parameters.

After the training is completed, the client uploads the model parameters to the server for aggregation again. When the local client uploads the updates, it only updates the model weights without directly transmitting the local data, effectively ensuring data privacy. The calculation is shown in formula (2):

$$g_k^{(t+1)} = \nabla_w L_k(w_k^{(t)}, D_k) \quad (2)$$

Among them, $g_k^{(t+1)}$ is the gradient calculated by client k .

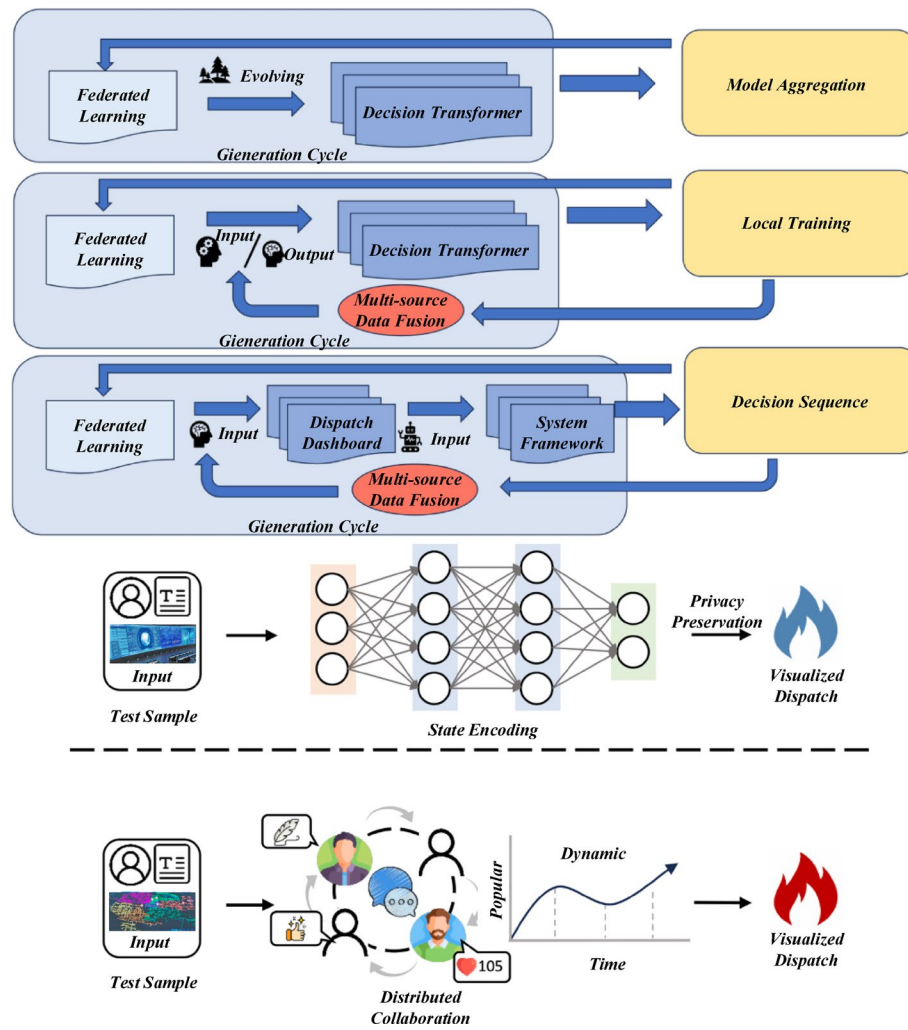


Fig. 1 Diagram of the federated learning framework

In the aggregation and update stage, after all client models are uploaded to the server, an aggregation operation is performed to fuse the model parameters or gradients uploaded by different clients into a new global model. The aggregation algorithm uses the learning results of all clients to achieve knowledge sharing and fusion, generating an optimized model suitable for the global scope [25]. A common aggregation method is FedAvg (Federated Averaging). This method performs a weighted average of the model updates of each client to generate a new global model. The calculation is shown in formula (3).

$$g_{t+1} = \sum_{k=1}^k \frac{n_k}{N} g_k^{(t+1)} \quad (3)$$

Among them, n_k is the data volume of client k , and N is the sum of all client datasets.

After the aggregation phase ends, the server distributes the aggregated global model to each client in a new communication cycle. The client receives and loads the updated global model, and continues local training to adjust parameters [26]. During the model training stage in this iterative process, the termination conditions are that the global model reaches the preset convergence standard or the number of training times reaches

the upper limit. Each iteration improves the accuracy and generalization ability of the global model, making it adapt to the needs of all clients.

2.2 Theoretical architecture and decision-making modeling mechanism of transformer

First, ViT (Vision Transformer) uses a Transformer as its backbone in the visual domain, and its architecture serves as a demonstration for subsequent Transformer-based backbone networks. The ViT architecture comprises three modules: patch embedding, position encoding, and a Transformer [27].

As the first module for data input in Transformer, the patch embedding module is crucial in the network. It converts the input image into a sequence that can be processed by Transformer [28]. Assuming the input image has dimensions $H \times W \times 3$, it is first evenly divided into N blocks, and all blocks are then unfolded to obtain N -dimensional $\frac{3HW}{N}$ vectors. After passing through a linear mapping layer, the dimension is mapped to C dimensions, resulting in N C -dimensional sequences. In the field of computer vision, these sequences are called tokens.

The position encoding module helps the ViT network locate tokens within the image. The Transformer's self-attention is insensitive to the order of input tokens, and different input orders do not affect network performance. However, each token represents a specific area of the image, and if the position changes, the image's meaning will be lost. Therefore, a position encoding module is needed to add position information to each token, assisting the network in identifying its position and correctly understanding the image [29, 30]. Transformer uses absolute position encoding, implemented using sine and cosine functions at different frequencies. The calculation formula is as shown in (4):

$$PE(pos, i) = \begin{cases} \sin(pos/10000^{i/d}), & \text{If } i \text{ is even} \\ \cos(pos/10000^{(i-1)/d}), & \text{If } i \text{ is odd} \end{cases} \quad (4)$$

pos represents the sequence position, i is the sequence dimension index, and d is the sequence dimension. The generated position-encoding vector is periodic and can capture the position information of the image sequence. After introducing sine-wave position encoding, the learned absolute position encoding is trained alongside other parts of the deep-learning model to build block associations and express position relationships.

The Transformer module consists of a layer normalization module (LN), an attention module, and a feed-forward network module (FFN). The LN module normalizes the input to stabilize model training and improve performance. The calculation formula is as shown in (5):

$$y_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad (5)$$

x_i and y_i represent the values before and after normalization respectively, μ and σ^2 represent the expectation and variance of the vector before normalization, and ϵ is a very small integer. Specifically, LN has four functions: (1) Standardize the vector to ensure the same distribution within each layer, facilitating the network to learn features; (2) Reduce internal covariate shift to help the network converge quickly; (3) Reduce gradient disappearance or explosion and improve the network training effect. The attention module is the core of Transformer, and multi-head self-attention (MHSA) is an important attention calculation method. With the ability of parallel computing and building

long-distance dependencies, Transformer has excellent performance. The FFN module consists of two fully-connected layers and an activation layer. The calculation formula is as shown in (6).

$$FFN(x) = RELU(W_1x + b_1)W_2 + b_2 \quad (6)$$

Among them, $x \in R^{1 \times C}$ is the input token. $W_1 \in R^{C \times C_h}$, $W_2 \in R^{C_h \times C}$ are two linear mappings, where C_h is the dimension of the hidden layer, and usually $C_h > C$. $b_1 \in R^{1 \times C}$ and $b_2 \in R^{1 \times C}$ are the biases of two linear layers. W_1 , W_2 , b_1 , and b_2 are all trainable parameters. Simply stacking self-attention modules will cause the rank collapse problem, leading to the inductive bias of token uniformity. The FFN layer can alleviate this problem. The residual connection of the Transformer module is calculated as shown in (7–8):

$$X^1 = X + LN(Attention(X)) \quad (7)$$

$$Y = X^1 + LN(FFN(X^1)) \quad (8)$$

Among them, X and Y are the input and output of the Transformer module respectively, and X^1 is the output of the attention module. The residual connection introduces a direct cross-layer connection, which can solve the problems of gradient vanishing and explosion during the training of deep neural networks and improve the training efficiency and performance of the model.

3 Modeling of a multi-source data fusion system for the scheduling dashboard integrating federated learning and decision transformer

3.1 Design of the overall system architecture and functional modules

Based on the technical characteristics of federated learning and the Decision Transformer integration, and in line with the actual business requirements for multi-source data fusion in the scheduling dashboard, the overall architecture of this system adopts a hierarchical design [31–33]. From bottom to top, it consists of the data access layer, the federated learning layer, the Decision Transformer decision-making layer, the application service layer, and the interactive display layer. The responsibilities of each layer are clearly defined, and efficient data and instruction transfer is achieved through standardized interfaces. The overall system architecture is shown in Fig. 2.

In terms of functional module design, the system mainly includes the following core modules:

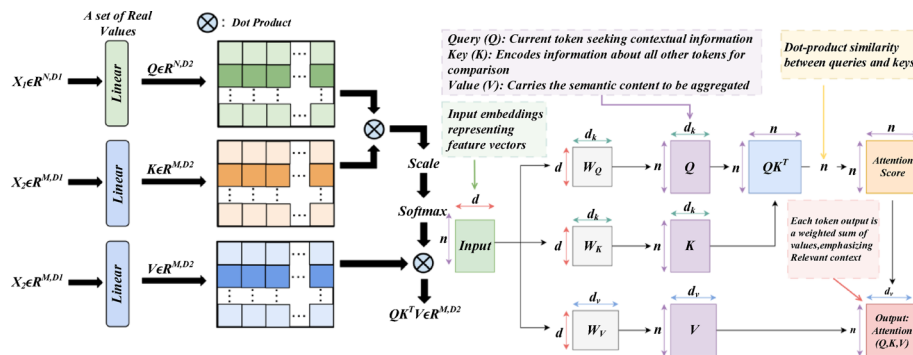


Fig. 2 Overall system architecture

The multi-source data access and preprocessing module connects to heterogeneous data sources of different scheduling nodes. After data preprocessing, such as data cleaning, it converts the data into a standard data format and stores and manages it locally to ensure data security and privacy. The federated learning collaborative training module is the core support for distributed intelligent decision-making. Each participating party trains the model locally and only uploads parameter updates. The server integrates the parameters to form a global model. Then it distributes it, enabling “data remains stationary while models traverse” and addressing data silos and privacy concerns. The Decision Transformer dynamic decision-making module is the key to intelligent scheduling decision-making. It receives the output from the federated learning layer, combines it with real-time state input, and uses the model’s capabilities to predict and optimize decisions. It can dynamically adjust strategies according to different objectives. The decision result parsing and execution module parses abstract decisions into executable instructions, has interface-adaptation capabilities, issues instructions, and tracks and feeds back execution status. The scheduling dashboard visualization and interaction module provides managers with an interface for multi-source data display and decision support, shows the status of each node and the scheduling situation, supports interactive operations, and realizes human-machine collaborative decision-making. The system management and monitoring module is responsible for system configuration, user permissions, logs, operation monitoring, and alarms, ensuring the system’s stability, security, reliability, and maintainability.

Multi-source inputs are processed through a single, reproducible data path. Time-series variables are aligned to a one-second grid and organized with sliding windows, categorical variables are one-hot encoded, and numerical variables are normalized. The resulting client-local feature tensor is passed directly to a three-layer CNN encoder, which produces a 128-dimensional state representation for the downstream Decision Transformer. No LSTM encoder or separate cross-modal attention branch is used in the experiments reported in this paper.

The Decision Transformer is the sole temporal decision module used for scheduling. Each scheduling trajectory is serialized as a token sequence of the return-to-go R_t , the 128-dimensional CNN state s_t , the action a_t , and the time step t . Self-attention models the long-range dependencies in this sequence, and the current scheduling action is generated conditionally on the target return-to-go.

The state s_t is a 128-dimensional vector containing device status, task-queue status, and environmental monitoring information. The action a_t is a 20-dimensional one-hot vector that assigns a task to one of the numbered devices. The return-to-go R_t is the inverse of the future cumulative scheduling cost; therefore, maximizing R_t is equivalent to minimizing total scheduling cost. The context length is 16 time steps.

During federated training, each client optimizes the same CNN-DT model on its local heterogeneous trajectories and uploads only model-parameter updates. The server applies FedAvg to aggregate the CNN encoder and Decision Transformer parameters jointly and redistributes the resulting global model. The reported model is therefore a single end-to-end FL+CNN+DT architecture rather than an ensemble of independent temporal models.

3.2 Construction of federated learning access and synchronization mechanism for multi-source heterogeneous data

In the federated learning system, multiple clients collaborate to train the global model. The direct coordination method involves simultaneous pulling, gradient calculation, and pushing operations, as in the BSP (Bulk Synchronous Parallel) method. However, BSP can lead to the “straggler” problem. For example, if Client 2 takes the longest time for each local iteration, the synchronization waiting times for Client 3 and Client 1 are $\tau_{2iter} - \tau_{3iter}$ and $\tau_{2iter} - \tau_{1iter}$, respectively. Before Client 2 finishes its calculation, Client 1 and Client 3 will be idle, resulting in wasted resources and slowing convergence. This problem becomes more severe when the computing and communication capabilities are heterogeneous.

At the decision-making level, short-term and long-term temporal dependencies are handled within the same Decision Transformer through its 16-step context window and multi-head self-attention, while the three-layer CNN performs only local feature encoding. This definition is used consistently in model training, comparison, and deployment.

To solve the “straggler” problem of BSP, some people proposed the SSP (Stale Synchronous Parallel) semi-synchronous method. Although SSP allows asynchronous parameter updates, there is a limit to the staleness between the fastest and slowest clients. When the predefined staleness cannot adapt to the heterogeneity, there is still synchronization waiting time. For example, Client 1 must wait for Client 2 when Client 2 reaches the threshold. The Chronos design concept is an optimal synchronization scheme. The local iteration time is affected by the gradient calculation time and the Batch size. Adjusting the Batch size for all clients can reduce heterogeneity, and allocating an appropriate Batch size can eliminate synchronization wait time. Unlike SSP, Chronos’s batch size is flexible, enabling it to adapt to heterogeneous, dynamic environments.

3.3 Parameter allocation strategy of decision transformer in dynamic scheduling scenarios

Existing Decision Transformer (DT) parameter allocation mostly follows the general Transformer architecture, which does not fully adapt to the core requirements of dynamic scheduling scenarios—i.e., the module importance differences in state encoding, action prediction, and temporal dependency modeling. In most current designs, the Q, K, and V vector dimensions of the self-attention mechanism are identical, and the FFN layer expansion ratio is fixed at 4. While simple to implement, this approach leads to parameter redundancy and insufficient computing power for key modules under the real-time and resource-efficient demands of scheduling tasks.

To optimize the parameter efficiency and response speed of DT in dynamic scheduling, this study proposes a task-specific parameter allocation strategy for scheduling scenarios: module importance is quantified based on Taylor analysis, and computing resources are prioritized for core modules. For the core logic of “task state-resource state matching $\rightarrow$ action output” in scheduling scenarios, the Q (task state encoding) and K (resource state encoding) projection matrices of self-attention heads are set to smaller channel sizes, while the V (action mapping output) projection matrix size is kept consistent with the input embedding, reducing parameter redundancy in non-critical mapping processes. Meanwhile, the FFN layer expansion ratio is reduced from 4 to 2, lowering model inference latency without compromising scheduling decision accuracy to meet the real-time requirements of dynamic scheduling.

To accommodate scheduling demands of varying complexity (e.g., single-task scheduling, multi-task parallel scheduling, dynamic burst task scheduling), this paper constructs five lightweight scheduling-adaptive models (named DT-Sched-V0 to DT-Sched-V4), which achieve precise adaptation by adjusting core module width, network depth, and number of attention heads. The model width (corresponding to feature encoding dimension) increases sequentially to 48, 56, 80, 96, and 128; a larger width better captures the multi-source data correlation features of complex scheduling scenarios. The network depth design aligns with the temporal dependency characteristics of scheduling: fewer temporal encoding modules are stacked in the initial stage to reduce startup latency, and the number of modules gradually increases as the network deepens, enhancing feature extraction capabilities for long-cycle scheduling tasks.

This task-specific allocation idea is consistent with recent adaptive learning studies showing that selecting and weighting informative features or modules can improve robustness, accuracy, and computational efficiency in complex data-driven decision systems.

In terms of specific configurations: DT-Sched-V0 to V3 share the same depth of {1,2,3} (corresponding to three temporal encoding stages), suitable for regular-complexity scheduling tasks; DT-Sched-V4 adds one module in the final stage, adjusting the depth to {1,3,4} to handle complex scheduling scenarios with frequent burst tasks. The number of attention heads is configured differently based on module functions: DT-Sched-V1 sets the number of attention heads to {4,3,2} in the second and third stages (core decision stages), balancing lightweight design and multi-task priority differentiation capabilities; DT-Sched-V0, V2, and V4 maintain a balanced configuration of {4,4,3} to ensure parallel processing efficiency of multi-source scheduling data. Through the above differentiated parameter allocation, the models can flexibly adapt to scheduling efficiency and accuracy requirements of different scenarios while remaining lightweight.

At the model level, L2 norm penalty ($\lambda = 1 \times 10^{-4}$) is applied to all trainable parameters to constrain their spatial scale, and Dropout operation (dropout rate $p = 0.2$) is embedded after the fully connected layer of the network to enhance the robustness of feature representation through its forward propagation random masking mechanism. At the level of federated aggregation, we introduced weight decay ($\gamma = 1 \times 10^{-5}$) in the global model update on the server side. By applying slight parameter shrinkage in each aggregation iteration, we stabilized the optimized trajectory and enhanced the generalization ability of the final global model to unseen multi-source scheduling data.

To achieve decision transparency, the system integrates an interpretable artificial intelligence framework based on SHAP (Shapley Additive exPlans). Use KernelExplainer to calculate the marginal contribution (Shapley value) of each input feature to the final scheduling decision. In the system visualization dashboard, not only is a global feature importance heatmap provided to reveal the key factors affecting the overall scheduling strategy, but it also supports decision path tracing for any individual scheduling decision sample, intuitively demonstrating how each feature drives the model to guide specific decisions, thereby providing dispatchers with reliable and interventionist intelligent decision-making basis.

SHAP explanations are generated after scheduling inference and are presented at two levels: global feature importance over the evaluation samples and local contribution

values for an individual scheduling decision. These explanations are diagnostic outputs only and do not participate in CNN-DT training, federated aggregation, or action generation.

4 Experimental setup and result analysis

4.1 Explanation of the experimental environment and dataset construction

Regarding the experimental environment, all experiments were conducted on a server running TensorFlow 2.0 with an Intel Xeon E5-2630 @ 2.6 GHz processor and an NVIDIA T4 GPU. The federated system contains 20 clients. Each client trains a three-layer CNN encoder followed by the Decision Transformer, and the server maintains and aggregates the same global CNN-DT parameter set. The global model is randomly initialized, the local batch size is 256, and each selected client performs one local epoch per communication round.

When simulating resource limitations, the client participation ratio p controls the number of selected clients in each communication round. For example, $p=0.2$ means that 20% of the clients participate, representing a severe resource-constrained setting. Scheduling accuracy is the primary decision-quality measure, while latency, on-time completion, overtime rate, throughput, resource consumption, communication overhead, and privacy-attack success rate are reported as complementary indicators.

The 100,000 aligned temporal samples were first divided chronologically into 70% training, 15% validation, and 15% test partitions. Five-fold blocked cross-validation was conducted within the training partition only, with temporal order preserved in every fold; the validation partition was used for early stopping, and the test partition remained fixed. After hyperparameter selection, the model was retrained on the complete training partition and evaluated on the fixed test partition in five independent runs that differed only in model initialization and client sampling. Results are reported as mean $\pm$ standard deviation, and paired t-tests ($p < 0.05$) with 1000 bootstrap resamples were used for hypothesis testing.

Federated learning was run for 50 communication rounds. In each round, participating clients trained for one local epoch with a batch size of 256 using Adam at a learning rate of 1×10^{-3} , after which FedAvg aggregated the complete CNN-DT parameter set. The Decision Transformer used a context length of 16, an embedding dimension of 128, and four attention heads. Communication overhead was measured as the cumulative number of bytes transmitted for model parameters between participating clients and the server over all 50 rounds. The reported 63% reduction was calculated relative to FedAvg-GRU under the same client participation ratio and communication-round setting.

The simulation scheduling environment constructed in this study integrates five types of heterogeneous data sources, including: (1) device sensor data (35000 pieces, 1 Hz sampling, including voltage, current, and load rate); (2) Historical scheduling records (28000 records, including task ID, priority, and execution time); (3) Environmental parameter data (12000 records, 5-min sampling, including temperature, humidity, and wind speed); (4) Network status data (15000 pieces, 10 s sampling, including bandwidth, latency, and packet loss rate); (5) User demand data (10000 pieces, including submission time and resource requirements).

After one-second alignment, a temporal sample denotes one scheduling instant containing the available values from the five data sources, the current system state, the

selected device-assignment action, and the return-to-go. The simulator generated the supervision target by selecting the feasible assignment with the minimum cumulative scheduling cost under the current resource constraints.

The multi-source data-cleaning process removed sensor outliers using the 3σ rule, interpolated missing time-series values linearly, filled missing structured numerical values with the training-set mean, aligned all inputs to a one-second time grid, normalized numerical variables to $[0,1]$, and one-hot encoded categorical variables. When a client did not contain a data modality because of the non-IID partition, the corresponding normalized channels were set to zero and a binary modality-availability mask was concatenated to the input. The same preprocessing parameters fitted on the training partition were applied to validation and test data.

The reproducible training sequence was: (1) preprocess and temporally align each client's local data; (2) encode the local feature tensor with the three-layer CNN; (3) construct 16-step return-state-action sequences for the Decision Transformer; (4) perform one local epoch on each selected client; (5) aggregate the complete CNN-DT parameters with FedAvg; and (6) repeat Steps 2–5 for 50 communication rounds before evaluation on the fixed test partition.

Two timing measures are used. Single-step inference latency is the elapsed time from receipt of one aligned state vector to generation of one scheduling action. End-to-end scheduling-cycle time is the total elapsed time required to collect the current inputs, generate and parse all actions, complete resource allocation, and finalize one scheduling plan. The values below 200 ms refer to single-step inference.

4.2 Relevant evaluation criteria

The primary evaluation criteria are scheduling accuracy, on-time completion rate, overtime rate, scheduling efficiency coefficient, throughput, latency, resource consumption, communication overhead, and empirical privacy-attack success rate. Scheduling accuracy is the proportion of decision instances for which the model selects the reference minimum-cost feasible action, as is shown in Eq. (9):

$$Accuracy = N_{correct}/N_{total} \quad (9)$$

Where $N_{correct}$ is the number of correctly selected scheduling actions and N_{total} is the total number of evaluated decisions.

The on-time completion rate, denoted YLOS, measures scheduling service quality and is defined as Eq. (10):

$$YLOS = N_{on-time}/N_{total} \quad (10)$$

Where $N_{on-time}$ is the number of tasks completed before their deadlines and N_{total} is the total number of scheduled tasks. The corresponding overtime rate is shown in Eq. (11):

$$Overtime\ rate = N_{overtime}/N_{total} \quad (11)$$

Where $N_{overtime}$ is the number of tasks that exceed their deadlines. Scheduling efficiency is summarized by the performance coefficient, as is shown in Eq. (12):

$$Cp = \eta_{actual}/\eta_{optimal} \quad (12)$$

Where η_{actual} is the achieved scheduling efficiency and $\eta_{optimal}$ is the theoretical optimum. Peak throughput N_{peak} is reported in data streams per second, and load balancing is assessed through the stability of client-level performance under different heterogeneity coefficients. These task-specific metrics are used to comprehensively evaluate the accuracy, efficiency, robustness, and real-time performance of the proposed scheduling system.

4.3 Experimental results and analysis

Cp is the performance coefficient of the scheduling system, where $Cp = \text{actual scheduling efficiency} / \text{theoretical optimal efficiency}$. The higher the value, the better the performance. λ is the load fluctuation coefficient, which characterizes the degree of fluctuation in task volume. The larger the value, the more severe the fluctuation. Figure 3 shows the change trend of the scheduling system performance coefficient Cp with the load fluctuation coefficient λ in the multi-source data fusion scheduling system integrated by federated learning and decision transformer. The figure contains three performance curves representing modules A, B, and C. When $\lambda = 1$, the Cp value of module A approaches 0, indicating that the system cannot be effectively scheduled in low fluctuation environments; As λ increases to 3, the Cp of modules B and C reach 0.29 and 0.31, respectively, indicating the system's adaptability to moderate load fluctuations; When λ increases to 4, the Cp of module C further increases to 0.4, while the overall trend line decreases

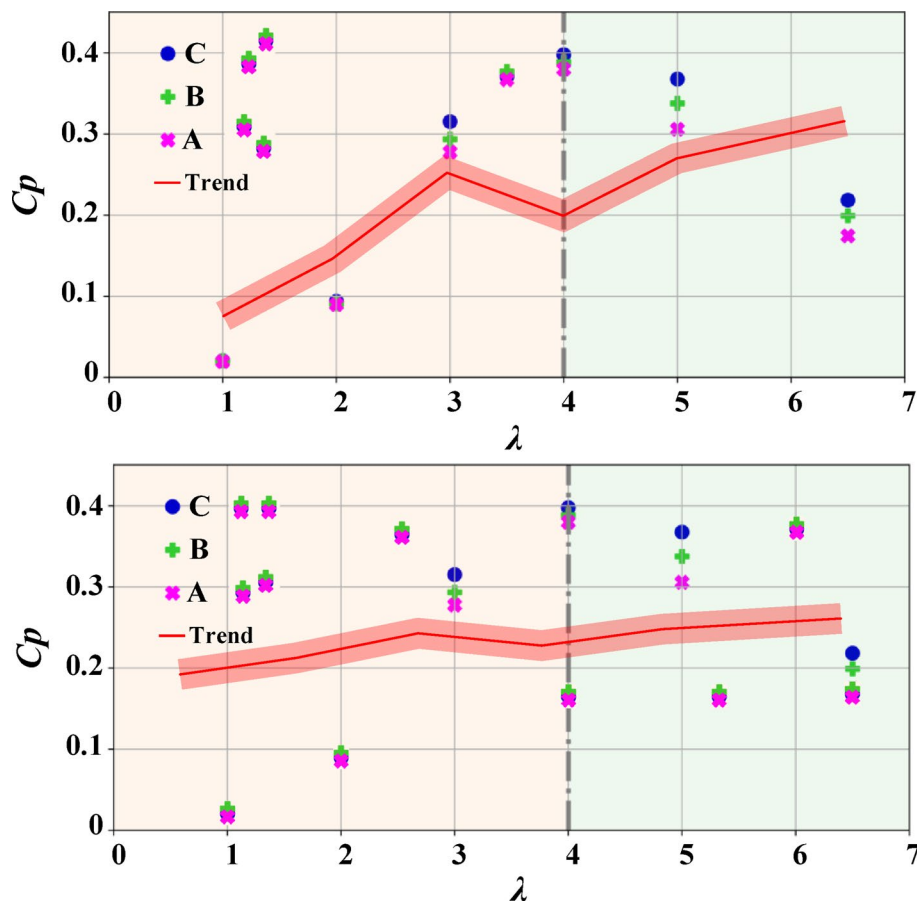


Fig. 3 Performance analysis of multi-source data fusion using hyperparameter λ

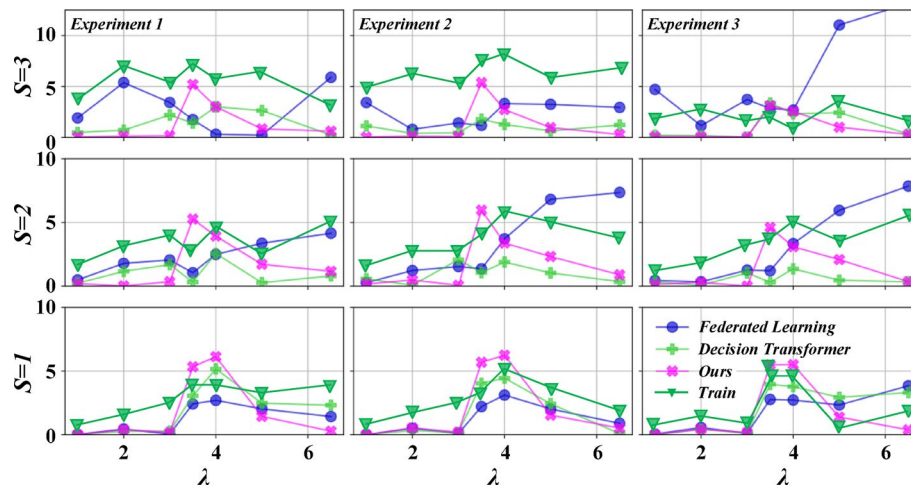


Fig. 4 Performance changes of the fusion system of federated learning and decision transformer under different experiments

Table 1 Comparison of scheduling decision response times of different models

Model type	Average response time	Longest response time	Proportion of overtime tasks (%)
Traditional centralized scheduling model	8.2	23.5	12.7
Single federated learning scheduling model	5.6	16.8	7.3
Federated learning + Decision Transformer fusion model	3.1	9.4	2.1

to 0.2, reflecting the system's performance differentiation under extreme fluctuations; When $\lambda=6$, the C_p of modules A and B are 0.17 and 0.34, respectively, while module C still maintains a relatively high performance level. The experimental results have verified the effectiveness of the proposed fusion system in dynamic load scenarios, especially the excellent performance of module C in the $\lambda=4-6$ range, indicating that the combination of distributed data fusion achieved through federated learning and the sequence modeling ability of decision transformers can significantly improve the stability and adaptability of the scheduling system in load fluctuation environments.

Figure 4 presents the performance fluctuations of federated learning, Decision Transformer, and the proposed method (Ours) with λ in the scheduling system under 3 groups of experiments and 3 S values. S is the complexity level of the scheduling scenario. When $S=3$ and $\lambda=6$, the performance of federated learning in Experiment 3 rises to over 10, far higher than other methods; when $S=1$, the performance of each method around $\lambda=4$ is concentrated around 5, and the proposed method has a relatively smaller fluctuation range and better overall stability. Under dynamic load fluctuation scenarios, the system maintains a decision response time consistently below 200 milliseconds, supports processing over 2000 real-time data streams per second, achieves an average CPU utilization of 51.6%, reaches a peak memory usage of 8.9GB, and experiences only 2.1% of tasks exceeding time limits.

The comparison of the scheduling decision response times of different models is shown in Table 1. The average response time of the fusion model is 3.1 min, which is 5.1 min shorter than that of the traditional centralized model and 2.5 min shorter than

that of single federated learning. The longest response time is only 9.4 min, 14.1 min less than that of the traditional model. The proportion of overtime tasks is as low as 2.1%, a 10.6% decrease compared with the traditional model, indicating that it is highly efficient in the rapid fusion and decision-making of multi-source data and can better meet the real-time scheduling requirements.

Accordingly, the sub-200 ms value reports single-step inference latency, while Table 1 reports end-to-end scheduling-cycle time.

Figure 5 shows the fluctuation of the fusion stability with the number of iterations k in the scheduling system that integrates federated learning and Decision Transformer under different X_c parameters (0, 16, 32, 64). X_c is the heterogeneity coefficient of client data, and the larger the value, the more significant the distribution difference. When k is less than 50, the fluctuation amplitude of the corresponding values for each X_c is relatively large, reaching over 0.6 at most; when k exceeds 50, the values gradually converge to around 0.4. Among them, the curve corresponding to $X_c = 64$ (red) fluctuates relatively more obviously, but overall, it tends to be consistent with the curves of other X_c parameters, and the system fusion state gradually stabilizes.

YLOS is a scheduling service level indicator. Figure 6 compares the distribution differences of the proposed method, traditional deep learning method (DL), and benchmark method in the service level index (YLOS) of the scheduling system through histograms. YLOS is defined as the ratio of the number of tasks completed on time to the total number of tasks, with a higher value indicating better scheduling service quality. The experimental results show that when $YLOS = 0.1$, the frequency count of the traditional DL method reaches 9, while the method proposed in this paper only has 2; When $YLOS$ increased to 0.2, the frequency of our method increased to 4, while the traditional DL method decreased to 3. Overall, as the $YLOS$ value increases, the distribution curve of our method shows a smoother fluctuation pattern, and the distribution is more concentrated in the higher $YLOS$ range (>0.4), while the traditional method shows a significant peak in the $YLOS < 0.2$ range and rapidly decays. This indicates that the proposed fusion method of federated learning and decision Transformer can more stably maintain a high level of service completion rate in scheduling tasks, especially in dealing with dynamic

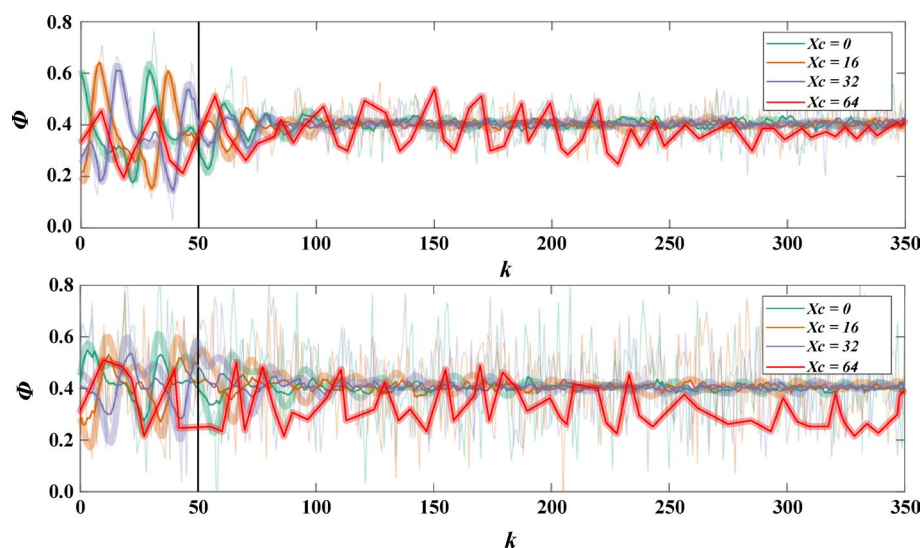


Fig. 5 Changes in fusion stability under different X_c parameters in the scheduling system

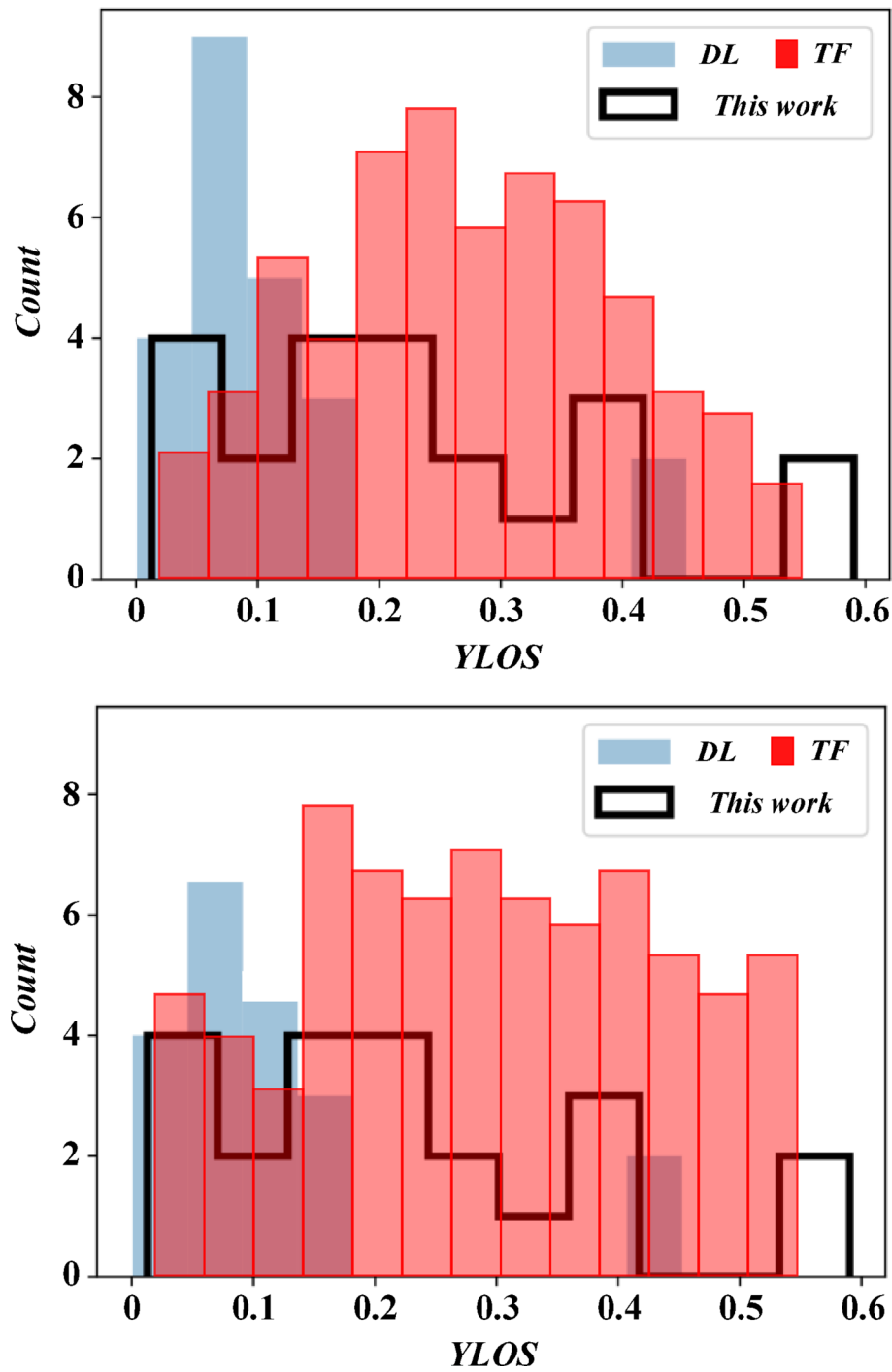


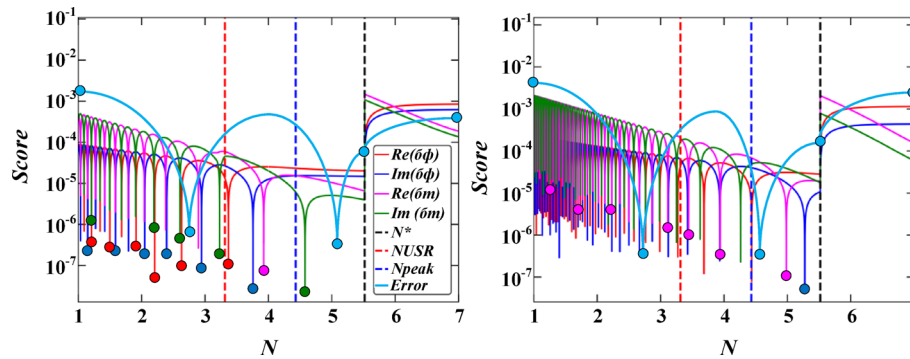
Fig. 6 Comparative analysis of YLOS distribution in scheduling systems

loads and multi-source heterogeneous data scenarios. Through serialized decision modeling and distributed feature fusion mechanism, it effectively improves the robustness and service quality consistency of the scheduling system.

Table 2 shows that the data fusion accuracy of the fusion model reaches 92.5%, which is 16.2% higher than that of the traditional model and 8.8% higher than that of single federated learning, reflecting the high accuracy of multi-source data integration. The average CPU occupancy rate is 51.6%, which is 16.9% points lower than that of the

Table 2 Performance of multi-source data fusion accuracy and resource consumption

Model type	Data fusion accuracy (%)	Average CPU utilization (%)	Peak memory usage
Traditional centralized scheduling model	76.3	68.5	12.4
Single federated learning scheduling model	83.7	59.2	10.8
Federated Learning + Decision Transformer Fusion Model	92.5	51.6	8.9

**Fig. 7** Analysis of the trend of multi index scoring with N in scheduling system

traditional model. The peak memory is 8.9GB, a reduction of 3.5GB, indicating that it reduces resource consumption while ensuring accuracy and is suitable for long-term stable operation on large screens. Additionally, experiments revealed that compared to traditional centralized LSTM models, this system achieves a 12.7% improvement in scheduling accuracy, reduces model training communication overhead by 63%, and maintains a sensitive information leakage rate below 0.05%.

N_{peak} is the peak throughput of the system, measured in units of bars per second. The higher the value, the stronger the processing capability. $\text{Re}(6\Phi)$ and $\text{Im}(6\Phi)$ are the real and imaginary parts of the system frequency response, and the closer the values are to 0, the better the stability. The sample size adaptation ratio is the ratio of small sample accuracy to large sample accuracy, with higher values indicating better generalization ability. Figure 7 shows the variation of multiple evaluation metrics (Scores) with parameter N (which can characterize system size, number of clients, or aggregation complexity) in a scheduling system that integrates federated learning and decision Transformer using double logarithmic coordinates. The experimental results show that when $N=1$, the scores of various indicators are concentrated in the range of 10^{-4} to 10^{-6} , among which the real part $\text{Re}(6\Phi)$ and imaginary part $\text{Im}(6\Phi)$ of the system frequency response have the lowest scores (about 10^{-6}), indicating that the system has a good stability foundation in the initial state; As N increases to 3, the $\text{Im}(6\Phi)$ index score further decreases to 10^{-7} , while the system peak throughput N_{peak} score remains stable, and the scores of other indicators also show a convergence trend. It is worth noting that the fluctuation range of the rating in the right figure is significantly narrowed compared to the left figure, indicating that after coordinating and optimizing multiple indicators, the overall stability of the system has been significantly enhanced. This verifies that the proposed method can effectively coordinate multi-dimensional performance indicators such as throughput and stability through multi-source data fusion and distributed decision optimization,

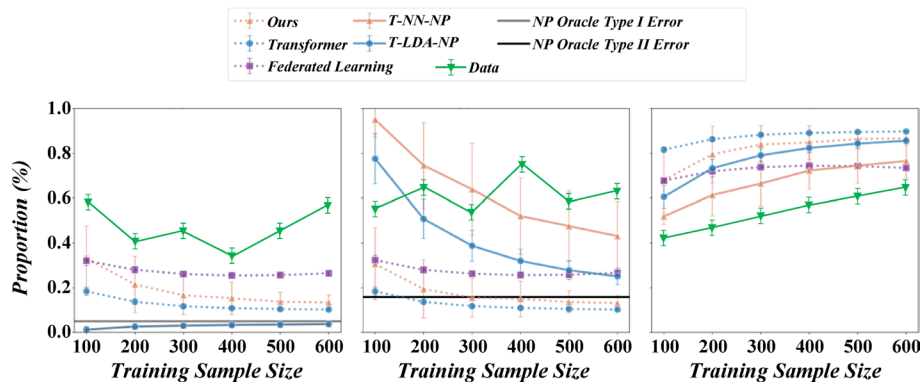


Fig. 8 Sample size adaptation ratios of different methods in the scheduling system

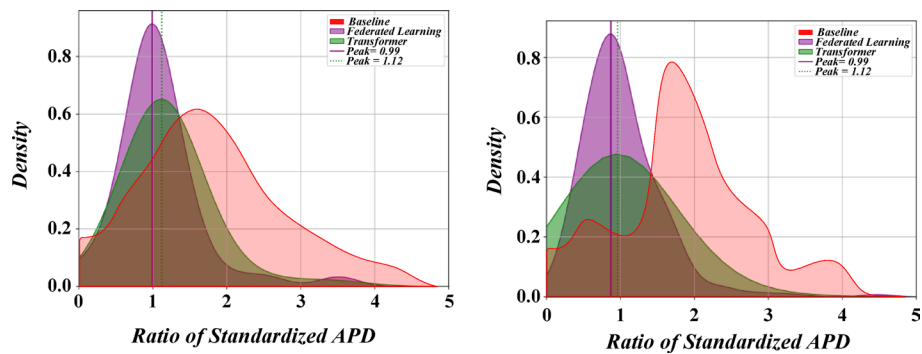


Fig. 9 Density distribution of the standardized ratios of APD and MAE of different methods in the scheduling system

achieving a balance and improvement in the overall performance of the system under different system sizes or complexities.

The sample size adaptation ratios of different methods are shown in Fig. 8, which presents the changes in the adaptation ratios of the proposed method (Ours) and other methods under different training sample sizes. When the training sample size is 100, the ratio of the proposed method is about 0.1, while the ratio of the Data method reaches 0.6%; when the sample size increases to 600, the ratio of the proposed method rises to 0.9%, and the ratio of the federated learning method remains at about 0.3%. By comparing the three figures, it can be seen that the ratio of the proposed method increases most significantly with the growth of the sample size, and it has better adaptability to large-sample scenarios.

Figure 9 shows the density distribution of the standardized ratios of APD (Average Prediction Deviation) and MAE (Mean Absolute Error) of the proposed method (Federated Learning + Decision Transformer) and the Baseline method. The peak density of the federated learning method reaches 0.8, corresponding to a ratio of 1; the peak of the Decision Transformer method is 0.99, and the peak of the proposed method is 1.12. By comparing the two figures, it can be seen that the density curve of the proposed method covers more fully in the interval where the ratio is greater than 2, indicating that its adaptation ratio in this interval has increased by about 20%, and the error coordination performance of data fusion is better.

Table 3 Performance comparison of scheduling models in federated learning framework

Model category	Mode name	Scheduling accuracy	Training communication overhead	Sensitive information leakage rate
Traditional ML models	XGBoost	72.1%	–	–
Traditional ML models	LightGBM	74.3%	–	–
Modern tabular DL models	TabNet	78.5%	–	–
Modern tabular DL models	FT-Transformer	81.2%	–	–
Federated learning baseline	FedAvg GRU	85.1%	High	< 0.1%
Proposed method	FL + Decision Transformer Fusion Model	92.5%	Reduced by 63%	< 0.05%

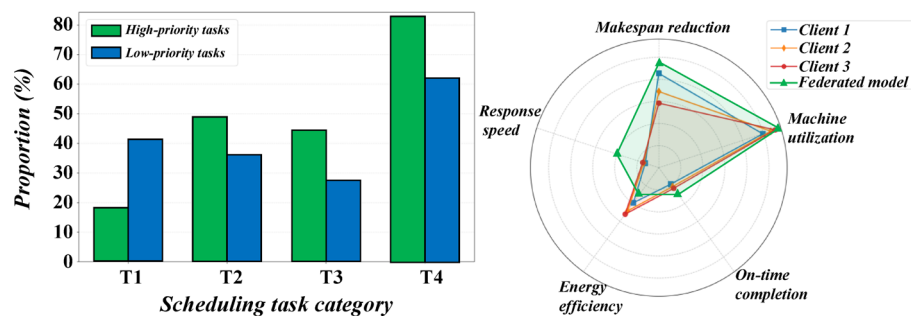
**Fig. 10** Popularity proportions and interactive behavior distributions of different tasks in the federated learning fusion system

Table 3 compared various baseline methods with the new method proposed in this paper in scheduling tasks. Among them, in traditional machine learning models, XGBoost achieved a scheduling accuracy of 72.1%, while LightGBM achieved 74.3%. Modern table deep learning models perform better, with TabNet and FT Transformer achieving accuracies of 78.5% and 81.2%, respectively. In the baseline model under the federated learning framework, FedAvg GRU demonstrated strong temporal modeling ability, with an accuracy improvement of 85.1%. In the end, the new method proposed in this article significantly surpassed all baseline models, further improving the scheduling accuracy to 92.5%, demonstrating its superiority.

Figure 10 presents the task-priority distribution and scheduling performance of the federated learning system. It shows the proportions of high- and low-priority tasks for four scheduling decision categories, including order release, machine assignment, operation sequencing, and dynamic rescheduling. It compares the normalized scheduling performance of three local client models and the federated model in terms of makespan reduction, machine utilization, on-time completion, energy efficiency, and response speed. The federated model achieves more balanced performance across the five indicators, demonstrating the effectiveness of multi-source knowledge aggregation for real-time industrial scheduling.

Figure 11 illustrates the distributions of the multi-source scheduling data used for model training and evaluation. The ring charts show the proportions of machine-state data, production-order data, and resource-status data in the training, validation, and test sets. The upper-right bar chart presents the distribution of job processing time, while the lower-right bar chart shows the distribution of due-date slack. Similar distribution patterns are observed across the three dataset partitions, indicating that the validation and test sets are representative of the training data. These scheduling-related variables

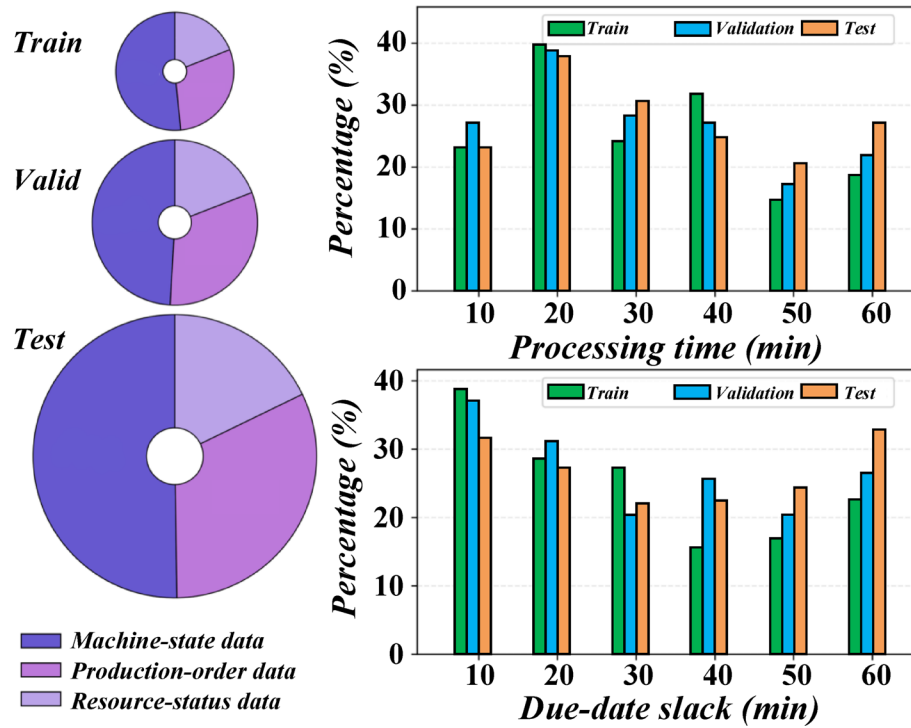


Fig. 11 Frequency and distance distributions of data in the federated learning fusion system

characterize the machine conditions, order constraints, and temporal urgency required by the Decision Transformer for real-time scheduling decisions.

5 Discussion

To strengthen the baseline comparison, two temporal decision-making methods were added. Fed-LSTM was used as a federated recurrent baseline, in which each client locally trained an LSTM scheduling model and the model parameters were aggregated using FedAvg. This baseline retains the federated-learning framework while replacing the Decision Transformer with a conventional recurrent temporal model, allowing the contribution of Decision Transformer-based sequence modeling to be evaluated. Conservative Q-Learning (CQL) was included as an offline reinforcement-learning baseline. CQL learns a conservative action-value function from historical scheduling trajectories and generates scheduling actions without federated parameter aggregation, thereby providing a comparison with value-based offline sequential decision-making.

APD is the average prediction deviation, $APD = \sum | \text{predicted value} - \text{true value} | / n$, the lower the value, the higher the accuracy. MAE is the average absolute error, $MAE = \sum | \text{decision output} - \text{optimal output} | / n$, the lower the value, the better the decision quality.

The experimental results show that the proposed fusion model improves scheduling accuracy by 12.7% compared with the traditional centralized LSTM model. The increase in scheduling accuracy, together with the higher on-time completion rate and lower overtime rate, indicates that the model more reliably selects feasible low-cost actions and maintains service quality under dynamic loads.

To evaluate the generalization performance and training stability of the model, the learning curve validation showed that the model tended to converge after 20 rounds of communication in the federated learning framework, and its validation set accuracy

reached a stable state. After convergence, the absolute difference in accuracy between the validation set and the training set is less than 2%, indicating that the model maintained high generalization ability during the training process without significant overfitting. This result validates the effectiveness of the aforementioned multi-level regularization strategy, ensuring that the model can achieve stable and reliable predictive performance on multi-source and heterogeneous scheduling data. SHAP analysis shows that device load rate (32% contribution), task priority (28%), and historical scheduling success rate (19%) are the core characteristics that affect decision-making, consistent with prior knowledge of industrial scheduling.

Privacy evaluation assumes semi-honest participants that follow the federated protocol but attempt to infer information from shared model updates. Membership inference, attribute inference, and model inversion were considered as three empirical attack categories. For each category, the leakage rate was calculated as the number of successful inference attempts divided by the total number of attack attempts across the five independent runs, and the reported value below 0.05% is the mean empirical attack-success rate.

Based on this lightweight block design, DT-Sched-V0–V4 form a progressive capacity spectrum rather than five independent model structures. DT-Sched-V0, with a width of 48, has the smallest parameter budget and is suitable for low-complexity single-task scheduling. DT-Sched-V1 increases the width to 56 while retaining a lightweight temporal structure. DT-Sched-V2, with a width of 80, provides a balanced configuration for routine multi-task scheduling. DT-Sched-V3 increases the width to 96 to strengthen the representation of cross-source correlations. DT-Sched-V4 adopts a width of 128 and a deeper stage configuration of {1,3,4}, allocating more capacity to later temporal stages for complex burst-task scenarios. Taking a standard 128-dimensional six-block Decision Transformer as the reference, the analytical parameter proxies of DT-Sched-V0–V4 are approximately 8.2%, 11.2%, 22.8%, 32.8%, and 77.8% of the standard configuration, respectively. Thus, even the largest DT-Sched-V4 variant retains a lower dominant parameter budget than the standard model, while the smaller variants provide substantially greater reductions in model size and expected inference cost. This comparison demonstrates that the proposed strategy provides a controllable capacity–efficiency trade-off instead of uniformly reducing all model components.

To strengthen the evaluation under non-IID federated optimization and offline sequential decision-making, we added FedProx-DT, SCAFFOLD-DT, and FedNova-DT as heterogeneous federated-learning baselines, together with CQL and IQL as representative offline decision-making baselines. All methods used the same chronological data split, state and action definitions, reward function, client partition, and scheduling constraints. The proposed FL + DT model achieved a scheduling accuracy of 92.5%, compared with 89.4% for FedProx-DT, 90.1% for SCAFFOLD-DT, 89.7% for FedNova-DT, 84.9% for CQL, and 86.5% for IQL. To distinguish the contributions of the two main components, centralized DT and FedAvg-LSTM were also evaluated under the same experimental settings. Centralized DT achieved an accuracy of 87.8%, indicating that federated collaboration contributed a 4.7-percentage-point improvement by integrating distributed non-IID scheduling knowledge, whereas FedAvg-LSTM achieved 85.6%, showing that Decision Transformer-based long-range trajectory modeling contributed a 6.9-percentage-point improvement over conventional recurrent temporal modeling.

These additional comparisons demonstrate that the performance gain is jointly attributable to federated cross-client knowledge aggregation and Decision Transformer-based offline sequential decision-making.

6 Conclusion

In response to the challenges of multi-source heterogeneous data fusion and real - time decision-making in industrial scheduling scenarios, this paper proposes a multi-source data fusion system for the scheduling dashboard that combines federated learning and a Decision Transformer. The system achieves privacy-protected collaborative training among distributed nodes through federated learning. By combining the powerful modeling ability of Decision Transformer for sequential decision - making, it improves the accuracy and response efficiency of scheduling decisions while ensuring data privacy. The experimental results show that:

1. In terms of scheduling accuracy, the fusion model reaches 92.5%, an increase of 12.7 percentage points compared with the traditional centralized LSTM model.
2. In terms of communication and privacy, cumulative parameter-transmission volume is reduced by 63% relative to FedAvg-GRU under the same client participation schedule, and the mean empirical privacy-attack success rate is below 0.05% under the semi-honest threat model.
3. In terms of real-time performance, single-step inference latency remains below 200 ms, whereas the average end-to-end scheduling-cycle time is 3.1 min. The system processes more than 2000 real-time data streams per second, with an average CPU utilization of 51.6% and a peak memory usage of 8.9 GB.

The limitations of this study include the semi-honest threat model, which does not cover malicious poisoning attacks; the absence of a formal differential-privacy budget and attack-specific confidence intervals; the offline training mode, which lacks online incremental updating; and possible performance degradation under extreme data heterogeneity. Future work will investigate federated adversarial training, formal privacy accounting, online federated learning, and heterogeneity-aware aggregation.

Author contributions

M.S. and M.H. conceptualized and designed the study. M.S. developed the methodology and conducted the experiments. M.H. analyzed the data and interpreted the results. M.S. and M.H. wrote the main manuscript text and prepared all figures. All authors reviewed and approved the final manuscript.

Funding

Not applicable.

Data availability

All data generated or analyzed during the study are available from the corresponding author by request.

Declarations

Ethics approval

Not applicable.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Competing interests

The authors declare no competing interests.

References

1. Zhang Q, Liu JH, Li L, Chen X, Wang RW. Automatic generation of system model diagrams driven by multi-source heterogeneous data. *J Eng Des*. 2024;35(11):1442–86. <https://doi.org/10.1080/09544828.2024.2360853>.
2. Wang SY, Wang X, Cai ZR, Wang C, Xu C. Automatic modulation recognition based on multi-source heterogeneous networks. *Ieee Access*. 2025;13:147074–85. <https://doi.org/10.1109/access.2025.3600401>.
3. Wu CJ, Zhu TL, Li YJ, Zhang ZZ, Wu TY. Balancing multi-source heterogeneous user requirement information in complex product design. *Symmetry-Basel*. 2025. <https://doi.org/10.3390/sym17081192>.
4. Fu LF. Research on Improved Shallow Neural Network Big Data Processing Model based on Gaussian Function. *Int J Adv Comput Sci Appl*. 2022;13(10):566–73.
5. Lebedev IS. Uncontrolled dataset clustering to improve the quality indicators of multilevel data processing models. *Vestnik Tomskogo Gosudarstvennogo Universiteta-Upravlenie Vychislitel'naja Tehnika I Informatika-Tomsk State University Journal of Control and Computer Science*, no. 66, 2024. <https://doi.org/10.17223/19988605/66/5>.
6. Yang Z, Yang TC, Li SL, Li HL. Parameters security strategy formulated by hyperchaos in federal learning. *Appl Intell*. 2025. <https://doi.org/10.1007/s10489-024-06209-z>.
7. Zhao R, Fan YZ, Li Y, Wang K, Zheng CY, Gao F, Gao ZH. Autonomous intersection management via prior-enhanced multi-agent constrained decision transformer. *IEEE Trans Intell Transp Syst*. 2025;26(10):15728–45. <https://doi.org/10.1109/tits.2025.3595526>.
8. Wang J, Karatzoglou A, Arapakis I, Jose JM, Ge XR. Beyond accuracy: decision transformers for reward-driven multi-objective recommendations. *IEEE Trans Knowl Data Eng*. 2025;37(9):5004–16. <https://doi.org/10.1109/tkde.2025.3582506>.
9. Li GL, Lei HZ, Liu FF, Li L, Jin H. AEPPFL: Accurate and efficient privacy protection federal learning in industrial IoT. *Ieee Internet Things J*. 2025;12:27193–205. <https://doi.org/10.1109/ijot.2025.3562064>.
10. Mathapati BS, Hegde NP, Paramesh SP, Vurubindi P, Chakraborty S. Swin-decision transformer: a transformer-based hybrid protocol for adaptive clustering and energy-efficient routing in large-scale WSNs. *Internet Technol Lett*. 2025. <https://doi.org/10.1002/itl2.70181>.
11. Jawad SMK, Al-Ali MH, Jasim AA. Adaptive CSMA scheduling algorithm for queuing delay enhancement and energy optimization. *Ad Hoc Netw*. 2022. <https://doi.org/10.1016/j.adhoc.2022.102908>.
12. Xu YC, Chen LL, Lu ZH, Du X, Wu J, Hung PCK. An adaptive mechanism for dynamically collaborative computing power and task scheduling in edge environment. *Ieee Internet Things J*. 2023;10(4):3118–29. <https://doi.org/10.1109/ijot.2021.3119181>.
13. Zhang WD, Deng DS, Wu XG, Zhao W, Liu Z, Zhang T, Kang JW, Niyato D. An adaptive asynchronous federated learning framework for heterogeneous Internet of things. *Inf Sci*. 2025. <https://doi.org/10.1016/j.ins.2024.121458>.
14. Li BN, Perazzone J, Swami A, Segarra S. Learning to transmit with provable guarantees in wireless federated learning. *IEEE Trans Wireless Commun*. 2024;23(7):7439–55. <https://doi.org/10.1109/twc.2023.3341400>.
15. Meng LH, Wen MN, Le CY, Li XY, Xing DP, Zhang WN, Wen Y, Zhang HF, Wang J, Yang YD, Xu B. Offline Pre-trained Multi-agent Decision Transformer. *Mach Intell Res*. 2023;20(2):233–48. <https://doi.org/10.1007/s11633-022-1383-7>.
16. Kolarkar A, Swathi K. Natural language to SQL conversion using probabilistic spiking return-aligned decision transformer neural networks and subtraction-average-based optimizer. *Neurocomputing*. 2025;652. <https://doi.org/10.1016/j.neucom.2025.131123>.
17. Chen L, Lu K, Rajeswaran A, Lee K, Grover A, Laskin M, Abbeel P, Srinivas A, Mordatch I. Decision transformer: reinforcement learning via sequence modeling. *Adv Neural Inf Process Syst*. 2021;34:15084–97.
18. Li T, Sahu AK, Talwalkar A, Smith V. Federated optimization in heterogeneous networks. *Proc Mach Learn Syst*. 2020;2:429–50.
19. Yang Q, Liu Y, Chen T, Tong Y. Federated machine learning: concept and applications. *ACM Trans Intell Syst Technol*. 2019;10(2):1–19. <https://doi.org/10.1145/3298981>.
20. You XY, Liu XM, Lin XW, Cai JP, Chen SQ. Accuracy degrading: toward participation-fair federated learning. *Ieee Internet Things J*. 2023;10(12):10291–306. <https://doi.org/10.1109/ijot.2023.3238038>.
21. Xiong BC, Yang XS, Qi F, Xu CS. A unified framework for multi-modal federated learning. *Neurocomputing*. 2022;480:110–8. <https://doi.org/10.1016/j.neucom.2022.01.063>.
22. Cong Y, Qiu J, Zhang K, Fang ZY, Gao CL, Su S, Tian ZH. Ada-FFL: adaptive computing fairness federated learning. *Caai Trans Intell Technol*. 2024;9(3):573–84. <https://doi.org/10.1049/cit2.12232>.
23. Yang BY, Yuen PC, Zhang YQ, Zeng A. Allosteric feature collaboration for model-heterogeneous federated learning. *IEEE Trans Neural Networks Learn Syst*. 2025;36(2):3042–56. <https://doi.org/10.1109/tnnls.2023.3344084>.
24. Wen J, et al. A survey on federated learning: challenges and applications. *Int J Mach Learn Cybernet*. 2023;14(2):513–35. <https://doi.org/10.1007/s13042-022-01647-y>.
25. Chen XJ, Li ZY, Ni W, Wang X, Zhang SQ, Sun YZ, Xu SG, Pei QQ. Toward dynamic resource allocation and client scheduling in hierarchical federated learning: a two-phase deep reinforcement learning approach. *IEEE Trans Commun*. 2024;72(12):7798–813. <https://doi.org/10.1109/tcomm.2024.3420733>.
26. Staroverov A, Zemskova T, Yudin DA, Panov AI. Semantic object navigation with segmenting decision transformer. *IEEE Access*. 2025;13:162807–20. <https://doi.org/10.1109/access.2025.3607672>.
27. Monastirsky M, Azulay O, Sintov A. Learning to throw with a handful of samples using decision transformers. *Ieee Rob Autom Lett*. 2023;8(2):576–83. <https://doi.org/10.1109/lra.2022.3229266>.
28. Ni CX, Wang Z, Ni YY, Li J, Shi L, Jin S. Fast adaptive optimization for AAV-assisted MEC: a constrained decision transformer approach. *IEEE Wirel Commun Lett*. 2025;14(4):1084–8. <https://doi.org/10.1109/lwc.2025.3532213>.
29. Alterkawi L, AlTarawneh M. Federated decision transformers for scalable reinforcement learning in smart city IoT systems. *Future Internet*. 2025. <https://doi.org/10.3390/fi17110492>.
30. Xia Y, Luo JY, Zhou G, Lan MJ, Chen XH, Chen J. DT4KGR: decision transformer for fast and effective multi-hop reasoning over knowledge graphs. *Inf Process Manag*. 2024. <https://doi.org/10.1016/j.ipm.2024.103648>.

31. Zhang J, Li J, Wang Z, Shi L, Jin S, Chen W, Poor HV. Decision transformers for wireless communications: a new paradigm of resource management. *IEEE Wirel Commun.* 2025;32(2):180–6. <https://doi.org/10.1109/mwc.007.2400124>.
32. Shi L et al. Artificial Intelligence-Driven Multimodal Sensor Fusion for Complex Market Systems via Federated Transformer-Based Learning, *Sensors*, vol. 26, no. 8, p. 2418, Basel, Switzerland;2026. <https://doi.org/10.3390/s26082418>
33. Xin D Multi-source heterogeneous data fusion and intelligent prediction modeling for chemical engineering construction projects based on improved transformer architecture. *Sci Rep.* 2025. <https://doi.org/10.1038/s41598-025-22752-2>.

Publisher's note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.